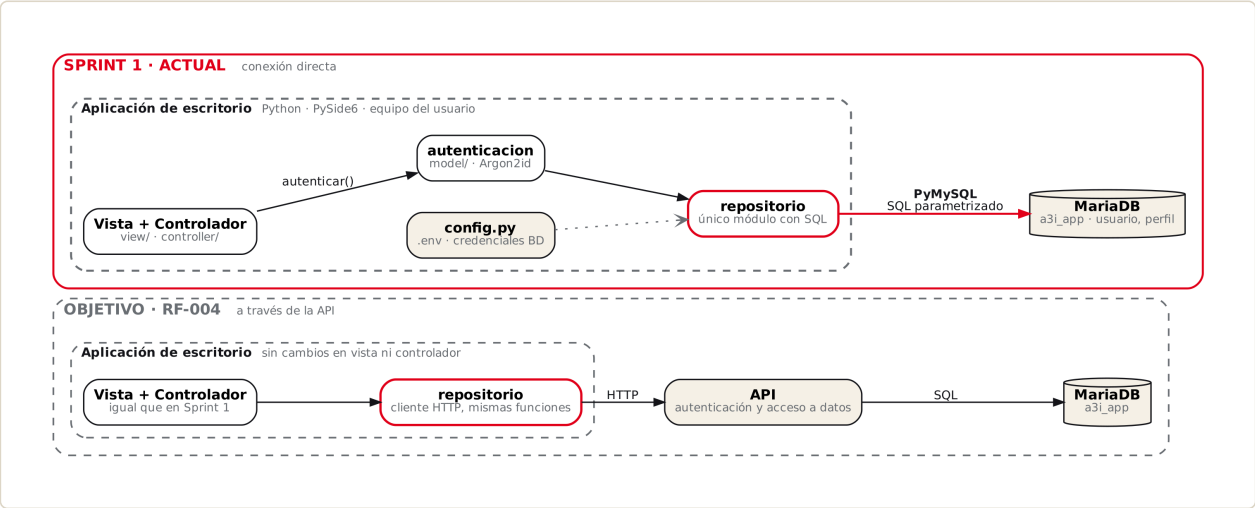
Diseño de la arquitectura aplicación–API–base de datos

Arquitectura mínima del Sprint 1 y su evolución prevista. En este sprint la aplicación de escritorio se conecta directamente a MariaDB; la API que pide RF-004 queda como deuda técnica conocida. El diseño aísla todo el acceso a datos en `model/repositorio.py`, de modo que el paso a la API solo cambia ese módulo.

1. Diagrama de arquitectura



En rojo, `repositorio`: el único punto que cambia al pasar de la conexión directa a la API.

2. Componentes (Sprint 1)

COMPONENTE	TECNOLOGÍA	RESPONSABILIDAD
Aplicación	Python, PySide6	Interfaz y lógica, organizada en MVC (RF-012). Se ejecuta en el equipo del usuario.
`repositorio`	PyMySQL	Único módulo con SQL. Consultas parametrizadas (RF-014). Envuelve los errores de MariaDB en `ErrorBaseDatos`.
`config.py`	python-dotenv	Lee del `.env` las credenciales de la BD y `MAX_INTENTOS_FALLIDOS = 5`. El `.env` no se sube al repositorio.
Base de datos	MariaDB, esquema `a3i_app`	Tablas `usuario` y `perfil` (RF-001), creadas con los scripts de `db/` (RF-022).

3. Decisiones

- **Conexión directa en el Sprint 1:** reduce el alcance para llegar al login funcional. Se documenta como deuda técnica frente a RF-004.
- **Punto único de cambio:** cuando exista la API, `repositorio.py` se sustituye por un cliente HTTP con las mismas funciones (`buscar_por_login`, `actualizar_intentos`, `bloquear_cuenta`, `actualizar_hash`). Ni vistas ni controladores cambian.
- **Seguridad:** el hash de la contraseña nunca sale del modelo, todas las consultas son parametrizadas (RNF-04) y los errores se muestran con un mensaje genérico (RNF-19).